

Self-GC：非研发 Agent 普惠时代的运行时上下文成本治理

Token 账单、上下文噪音与长程 Agent 的 Active Context 管理

演讲人：郝栩彬

公司 / 职位：小红书质效研发部 — AI工程架构师

AiCon

全球软件开发技术大会



目录

01

Agent 普惠后的成本账

非研发场景、输入 Token、模型路由与行为退化

02

成本治理指标与策略

Avg Input Tokens、Compaction Reset、三类工程策略

03

Self-GC 运行时机制

ContextObject、Plan / Commit、生命周期动作

04

Context Injection 与数据飞轮 旁路 Copilot、Session 清洗、Benchmark / SFT / RL

05

评测结果与 Workload 洞察

Hard Set、A/B、办公与 Coding 场景差异

06

总结与展望

三点结论、边界与开源方向

Agent 普惠后的成本账

从非研发使用规模到长上下文账单硬着陆



白领 Agent 普惠：成本问题开始变成企业问题

AGENT ADOPTION MAKES CONTEXT COST A BUSINESS PROBLEM

起源

- 过去 Agent 体感最强的场景多在 Coding Agent；白领人群的渗透相较于25年初 依然是 豆包、元宝、DeepSeek 为主进展缓慢，直到今年小龙虾的爆火，XX Work XX Claw 大量涌现。
- 白领任务步长相对短，但用户基数更大、使用频率更高，配套工具大量未经专门优化，工具 tokens 效率低，推理流量会被放大到新的量级。
- 此外，Coding 任务大多有交付物 / Diff Patch，围绕编码的 Git、编译环境、存储 均是强鲁棒性的运行环境；与此对立白领任务探索空间更广，浏览器、APP、Office，存在大量的无效 token 被浪费

规模放大逻辑



Token 管理不再只是“别爆窗口”，而是企业级 Capacity Planning。

长程白领 Agent 的输入吞吐计算

CAPACITY PLANNING AND MODEL ROUTING

1000人企业的并发容量账

- 按峰值并发计算：千人公司，同一时间 100 名员工运行 OpenClaw
- AVG TPM 1千人公司 = 1 kw tokens / min**
- 每个连续任务平均 5 次模型调用，非定时任务 瞬时输入吞吐可达 2~3 kw Input tokens / min。

Backbone Model	每分钟输入价格	每分钟输出价格	8h成本	22天成本
海外旗舰模型	\$7.45	\$447	\$3,576	\$78,672
qwen3.7 plus	\$0.97	\$58	\$464	\$10,208
deepseek V4 pro	\$0.53	\$31.54	\$252	\$5,551

顶级模型仍然绕不开，因此需要通过 Self-GC 缩小它每次读取的上下文前缀。

为什么不能全部切到低价模型

- Browser-First 任务离不开多模态：截图、交互态 DOM、Office 报告，第一眼看错会一路跑偏。
- 用户意图经常模糊，模型需要澄清、追问并避免幻觉执行。
- 长步数工具链要求 XML / JSON / 工具参数 / 规划格式稳定，任何一环崩掉任务都会变形。
- 生产上更可行的是多模型混合路由：轻量步骤用低价模型，关键规划和视觉理解回到顶级模型。

LONG CONTEXT BEHAVIOR DEGRADATION

图 1-a 复读机退化

图 1-b CoT 标签逃逸

图 1-c 工具调用死循环

- 轮次一长、噪声一多，即便顶级模型也会出现“长程脑雾”。
- 工具调用最麻烦的是“行动惯性”：模型 reasoning 里意识到应停止，但动作层继续发同一个 exec 请求。
- 一次这样的失控，足够让单个会话的 Token 消耗断崖式上涨。

AiCon 探索 AI 应用边界

姚顺雨：性价比不是单点价格，而是系统性问题

LONG CONTEXT BEHAVIOR DEGRADATION

采访中的关键判断

“

很多时候，高性能模型反而更省钱，因为它能一次把事情做对，既节省 Token，也节省人力成本。

”

- 性能是性价比的核心：一次做对，比低价反复试错更重要
- 成本优化是体系工程：模型架构之外，还包括长文管理、脚手架优化、任务拆分
- 中杯模型价值：在于高频简单任务上强鲁棒性

结论：成本优化不是“全面切小模型”，而是“高性能模型兜底 + 小模型高频执行 + 上下文持续压缩”的系统工程。

对白领 Agent 的工程启示

- 顶级模型负责复杂任务与兜底：处理复杂推理、意图模糊和失败恢复
- 中小模型负责确定性子任务：典型的 cron job、workflow、明确的执行 Skill
- 白领任务差异性上下文管理：面对白领任务不能简单沿用面向 Coding Agent 设计的上下文管理机制

成本治理指标与三类工程策略

从 Avg Input Tokens 到 Active View 的持续治理



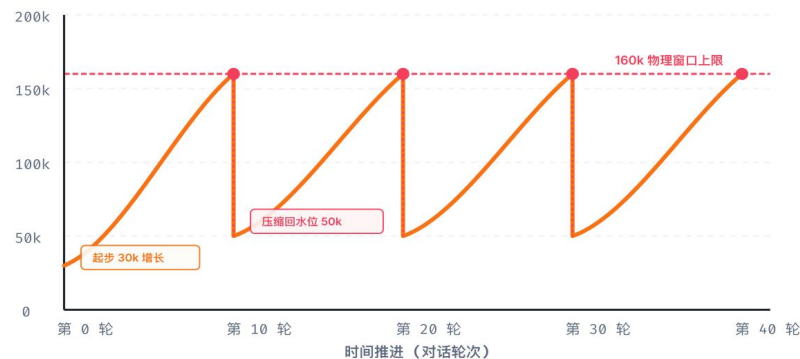
成本治理主指标：Avg Input Tokens

COMPATION RESET IS NOT ENOUGH

为什么选这个指标

- Total Tokens 会被任务量和日活波动带偏，不能直接体现单次调用成本。
- Output Tokens 在白领 Agent 中通常很小，只看输出意义有限。
- “有没有爆上下文窗口”只是系统别崩的底线，不是财务上的精细治理。
- Avg Input Tokens 直接衡量单次模型调用平均要读多少输入，更接近成本源头。

图二：物理 Context Window 限制下的被迫重置 (Compaction Reset)
(线下阴影面积 = 累计消耗的 Tokens)



图二：物理 Context Window 限制下的被迫重置 (Compaction Reset) 曲线。上下文从 30k 起步平缓增长，一旦触碰 160k 物理上限，便被强制重置压缩回 50k 水位线，呈周期性大起大落。

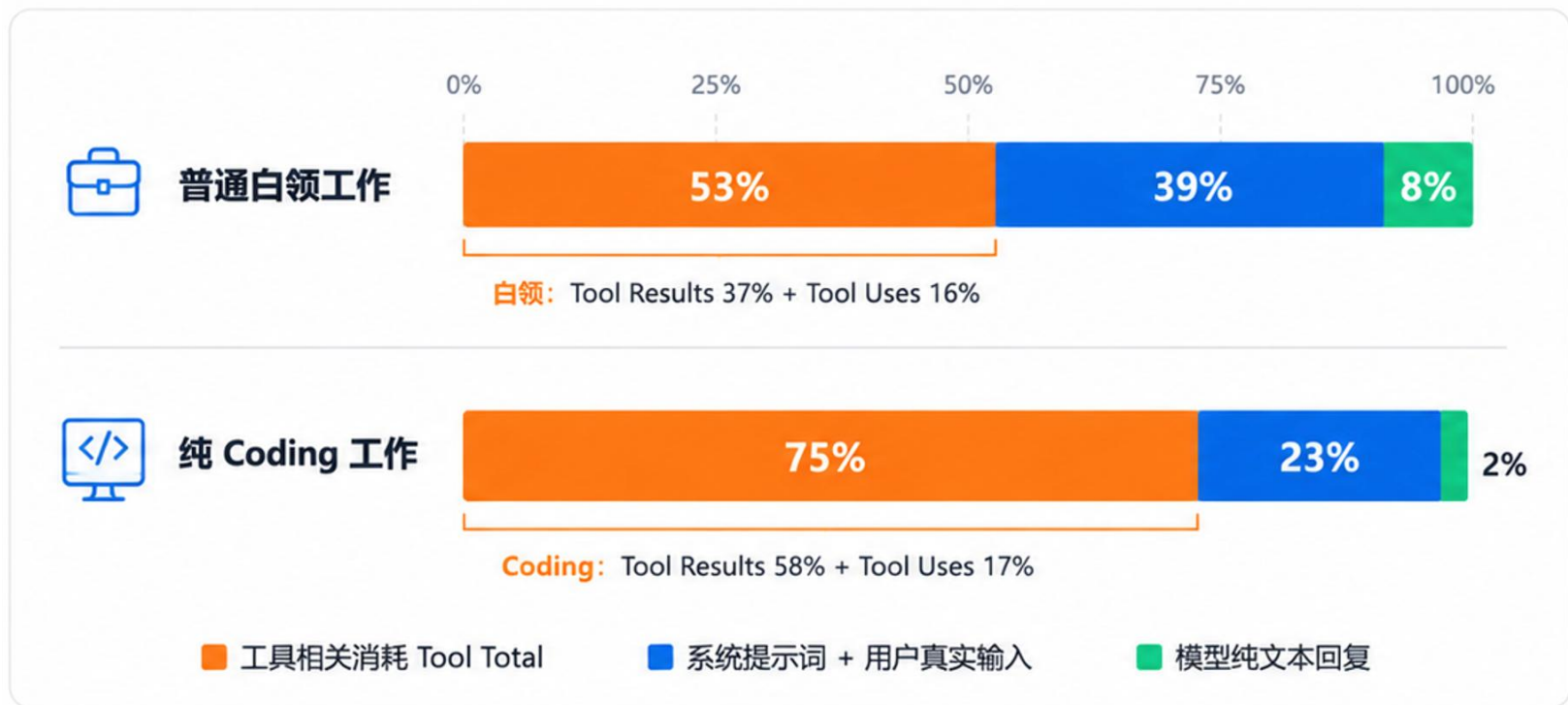
图二：物理 Context Window 限制下的增长曲线

等窗口撞满再 Summary 只能避免溢出，无法真正降低长期输入成本。

Context 中有什么

THREE TOKEN COST GOVERNANCE STRATEGIES

白领vs Coding:上下文Token消耗占比



💡 关键洞察

- Coding 场景中，上下文更被工具流主导。
- 白领场景仍保留较多 System / User 底噪。
- 模型自然语言回复占比都不高，Coding 最低。

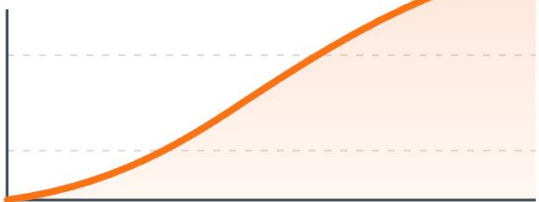
结论:Coding更像“工具驱动型上下文”，白领更像“工具+文本混合型上下文”

降低 Avg Input Tokens 的三类工程策略

THREE TOKEN COST GOVERNANCE STRATEGIES

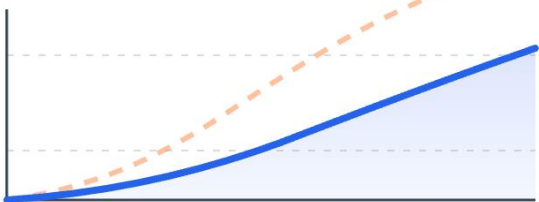
Baseline: 不裁剪自然增长

曲线持续抬升，线下面积最大



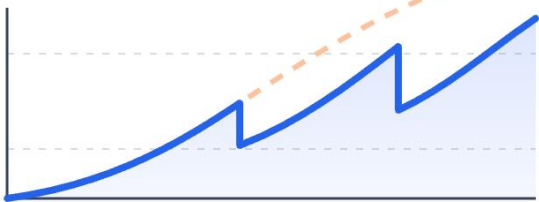
降低增长斜率

源头少产生 token，曲线整体变平



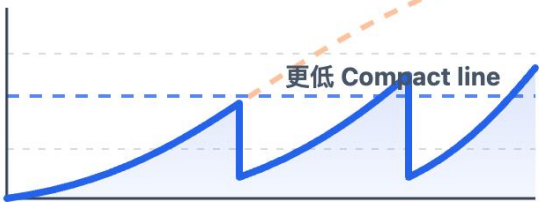
过程裁剪

增长仍存在，但周期性下探，均值下降



激进压缩 / 高频 Compact

强 reset 保持低位，但更依赖有损摘要



策略说明

- 策略一：降低 Context 增长斜率。源头治理，减少冗余工具输出和无效 loop，是优先级最高的无损优化。
- 策略二：过程裁剪 In-Run Pruning。任务运行中定期清理过程噪声、中间检索结果和已 resolved 状态，Self-GC 主要解决这一类问题。
- 策略三：激进 Compact。更早、更频繁触发 Summary，但 Office / Web 里的 URL、表格行和页面状态一旦被压扁就难以恢复。

合理策略不是单次压缩，而是在运行中持续治理 Active View。

策略一：让模型输出裁剪指令，先修工具结果

TRIM INSTRUCTION AS THE CONTROL PLANE

1. 原始工具输出

stdout / 编译日志 / API JSON 先不进入主 Agent transcript.

2. 小模型只输出裁剪指令

输入：用户问题 + 对话历史 + 原始工具结果。
输出不是摘要，而是结构化 selector。

3. 系统执行指令重建结果

主上下文只 append 被筛过的 Shaped Result，增长斜率从源头下降。

text → {"mode": "line_ranges", "keep": [[start, end]]}

json → {"mode": "json_paths", "keep_paths": ["\$.path[...]"]}

例子 A：纯文本日志

长 stdout 中只有最终统计表对用户问题有用。

231 → 14

行数下降 94%

Before 原始 stdout

231 行

```
1-2 openpyxl warning...
3-16 10 分类统计表
17-83 品牌 A TOP3 笔记正文...
84-151 品牌 B TOP3 笔记正文...
152-231 大量 URL / 原文 / 重复字段...
```



After Shaped Result

14 行

```
drop 1-2 warning
keep 3-16 分类统计表
drop 17-231 原始笔记列表

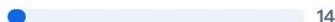
结果进入 transcript:
只留下可回答问题的表格
```

Before



231

After



14

例子 B：纯 JSON 响应

Pipeline API 只需要失败 stage，成功 and skipped 对象不进上下文。

8 → 3

对象/分支保留

Before API JSON

完整响应

```
{
  "pipeline": "failed",
  "checkout": "success",
  "install": "success",
  "unit_test": {"status": "failed"},
  "build": "skipped",
  "artifacts": {...}, "env": {...}
}
```



After Shaped Result

相关 JSON

```
{
  "pipeline": "failed",
  "branch": "main",
  "unit_test": {
    "error": "...",
    "logs": [...], "summary": {...}
  }
}
```

Before



8

After



3

核心价值：用“裁剪指令”控制工具结果进入上下文的形状，而不是等上下文变胖后再补救。

策略二：过程中裁剪，持续治理Active View

IN-RUN PRUNING BEFORE EACH MODEL CALL

1. Full Session Store

客户端 / harness 保留完整 transcript，作为审计与恢复底座。

2. Cheap Pruning Pass

0 API 操作优先：大结果落盘、旧 tool_result 占位、消息中段 snip。

3. Active View Projection

LLM 只看到最近工作面、summary pointer、必要的恢复线索。

参考方案抽象：便宜的先跑，贵的后跑

Hermes / Claude Context Editing / Claude Code compact 的共同模式。

0 API → 1 API

按压力升级

L1 大结果落盘

tool_result_budget

单条工具结果太大时，只留预览和文件指针。

500KB stdout
→ preview + path

L2 旧结果占位

tool result clearing

老的文件内容、搜索结果、命令输出替换为 placeholder。

[Old tool output cleared]

L3 消息中段裁剪

snip / keep tail

保留开头约束和最近工作面，中间旧轮次折叠。

head + [snipped]
+ recent tail

L4 摘要 / 应急

compact / reactive

阈值仍超时才让模型总结；API 拒绝时做 fallback。

summary + tail
on threshold

Before / After：输入视图变薄

过程裁剪不是改 full history，而是改下一轮 LLM 看到的 active view。

95K → 45K
示意下降

Before Active View

~95K

system + first request

keep

old search results

18K

old file contents

27K

thinking blocks

15K

After Active View

~45K

system + first request

keep

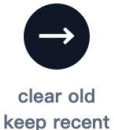
search results pointer

fold

file path + preview

fold

structured progress summary sum



Full transcript 保留在本地；LLM input 只保留“当前能推进任务”的工作面。

过程裁剪的关键：不是一次性压缩到底，而是在 Agent loop 中持续维护更小、更准的 Active View。

策略三：训练模型自主压缩，把 Compact 原生集成

TRAINED SELF-SUMMARIZATION INSIDE THE AGENT LOOP

1. Long Rollout

Agent 持续探索、执行工具、积累超出窗口的长轨迹。

2. Trigger Self-Summary

接近固定 token trigger 时，插入合成总结请求和短暂思考空间。

3. Continue With Condensed State

摘要携带目标、计划状态、剩余任务、关键路径和错误修复。

4. Reward The Whole Chain

最终任务奖励同时作用于行动 token 和让行动成功的摘要 token。

Cursor Self-summary

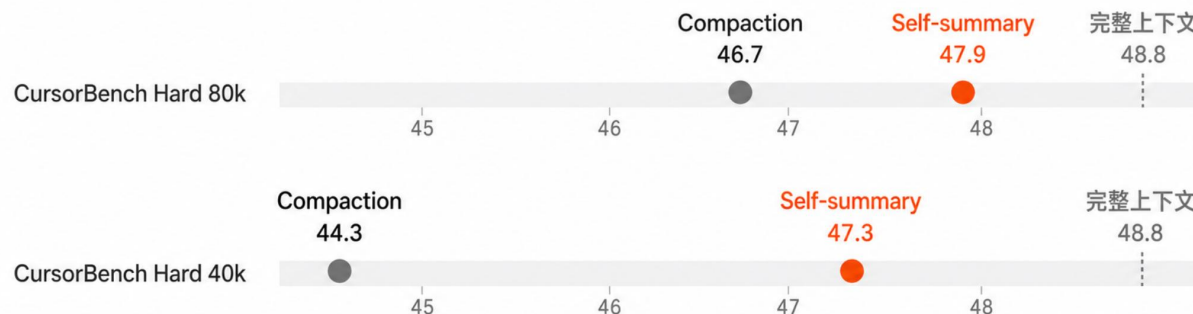
Cursor 把压缩从 prompt 技巧变成训练目标。

当长程任务接近上下文上限时，模型自己生成 condensed context，并带着它继续完成任务。

后续任务结果会反向训练 summary tokens，让模型学会保留未来有用的信息，而不是简单截断历史。

图中结果：Self-summary 仅用约 1/5 tokens，却更接近 full context 精度。

Self-summary 在更少 token 下，接近完整上下文精度并显著优于标准 Compaction



含义：模型学到如何把长程状态压缩成可继续工作的摘要，而不是简单丢弃历史。

Self-summary 仅使用 1/5 tokens



模型自己学会交接长程状态；Self-GC 是这个能力成熟前的运行时治理补位。

规则裁剪的边界：Coding 舒适区 vs 白领任务

RULE-BASED PRUNING WORKS UNTIL CONTEXT BECOMES HETEROGENEOUS

同样是过程裁剪，Coding 的外部恢复基座强；白领任务的关键状态更碎、更难靠规则识别。

Coding：规则裁剪的舒适区

- 工具更幂等：编译、测试、grep、git diff 可重复执行
- 状态更结构化：文件树、commit、patch、blame 自带记忆
- 误裁后可恢复：失败日志和上下文可重新生成
- 启发式影响面较小：删旧日志通常不破坏任务目标

规则：保留最新日志 / 折叠旧工具结果，通常够用

白领任务：规则裁剪容易误伤

- 工具输出少但更杂：URL、截图、表格行、Office 片段混在一起
- 状态不可幂等：网页/APP/文档状态变化后很难复现
- 关键锚点很小：一个链接、行号、人工确认会影响后续动作
- 固定规则看不出未来依赖，容易不可逆裁剪

需要：判断对象未来价值，而不是只看时间和类型

结论：规则裁剪适合清理“可重放噪声”；白领 Agent 上下文编辑需要引入模型判断

04

Self-GC 运行时机制

把 Transcript 从文本拼接升级为 ContextObject 生命周期管理



从 JVM GC 类比理解 Self-GC

CONTEXT AS RUNTIME HEAP

JVM 垃圾回收直觉

- 在 JVM 中，堆内存管理本质上是在管理对象生命周期：哪些对象还活着，哪些对象已经没人引用。
- 对象标记 Marking：从 GC Root 出发沿引用链查找，能找到的是活对象。
- 对象整理 Compacting：将存活对象向一侧整理，形成连续空闲空间，降低内存碎片。

对应到 Agent 上下文

- 长程对话历史就是 Agent 运行时的虚拟 Heap。
- 如果只是文本拼接或阈值截断，本质上是在语义边界中间随手切一刀。
- 要安全缩上下文，必须先给文字“上户口”：对象有身份、有边界，后面才能被标记、折叠、遮罩或删除。

图四：JVM 垃圾回收（Mark-Sweep-Compact）示意图

1. Before GC (堆内存碎片化)



2. Marking Phase (从 GC Root 标记活跃对象)



3. Compact Phase (向左压缩整理，腾出连续空闲内存)



图四：JVM 垃圾回收机制的 Mark-Sweep-Compact 整理流程（将存活块往左物理搬移对齐）

ContextObject: 让上下文可索引、可恢复、可治理

FROM TRANSCRIPT TEXT TO CONTEXT OBJECTS

看待上下文的方式需要改变

- 不要再把 Transcript 当成一整条越拼越长的 Message text。
- 应把上下文拆成一组有身份、有边界、可索引的 Context Objects。
- 一个用户请求可以是 conversation:user:k；一次工具读取或执行可以是 function:tool:n。
- 只有对象有了 ID 和边界，模型才可能判断“这个对象后面还会不会用到”。
- 对象化之后，active view 不再等于完整历史，而是对历史对象的一次可控投影。

运行时对象视图



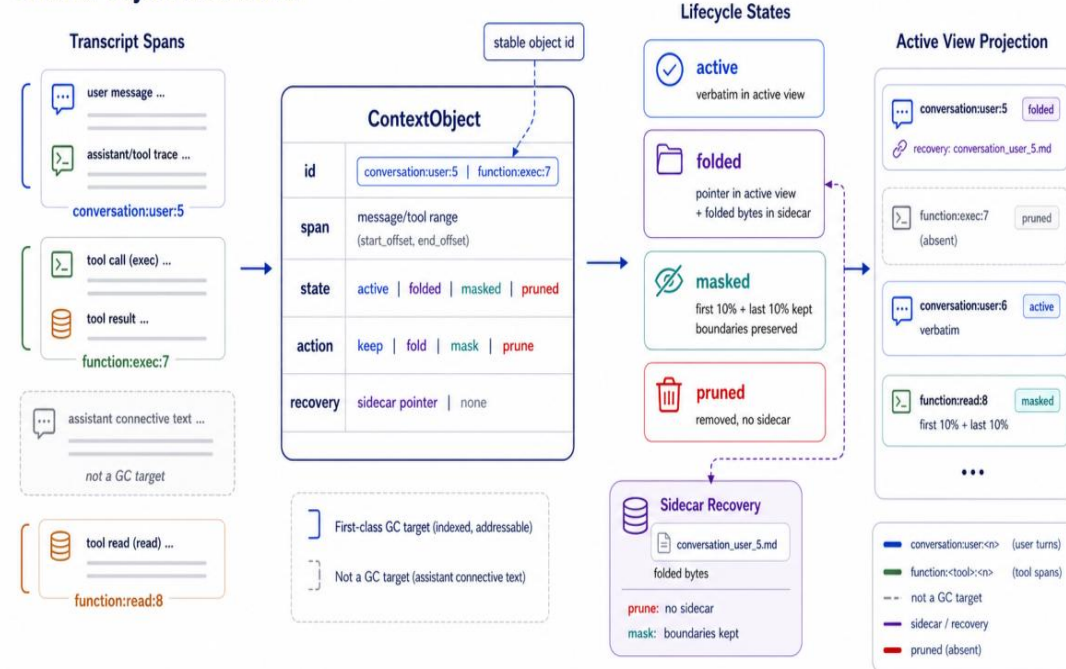
对象类型	示例	治理意义
用户轮次	conversation:user:12	可追踪用户意图与最新需求
工具执行	function:exec:8	可区分失败重试和有效动作
读取结果	function:read:14	可保留结构边界并做遮罩
恢复指针	recovery:conversation_user_1 2.md	折叠后仍可回填原始证据

Self-GC 的核心前提是上下文对象化；否则就无法做到精确折叠、遮罩或删除。

ContextObject 生命周期状态流转

DATA MODEL AND ACTIVE VIEW PROJECTION

Context Object Data Model



状态含义

- active: 对象仍以原文或主要内容出现在 Active View 中。
- folded: Active View 中仅保留可寻址指针，完整原始内容存放在 Sidcar Recovery。
- masked: 保留边界和少量片段，减少 token 占用同时保留结构线索。
- pruned: 从 Active View 中移除，通常用于失败重试、无效中间尝试等低价值对象。

给上下文“上户口”：基于 turn_id 的元数据标记

CONTEXT OBJECTIFY VIA TURN_ID METADATA

对上下文建立索引

- 在 User Input 和 Tool Result 中，以元数据格式显式注入唯一 turn_id。
- 将原本越拼越长的 Transcript 文本切割为具有强物理边界的 ContextObject。
- 只有对象有了明确的身份标识与边界，Planner 和控制层才可能做精确折叠与删除。

User turn_id 注入协议示例 (JSON Header)

```
[User]:  
Sender (untrusted metadata):  
{  
  "label": "webchat",  
  "turn_id": "conversation:user:1"  
}
```

这个文件是啥代码?
[Assistant]: 让我先读一下 main.py

Tool turn_id 注入协议示例 (Tool result)

```
[Assistant Tool Call]:  
{  
  "function": "read",  
  "filepath": "main.py",  
}  
[Tool Result]  
[File Content (src/main.py)]:  
def main():  
    print("Self-GC System Running...")  
  
if __name__ == '__main__':  
    main()  
<turn id="function:read:1" />
```

旁路 Fork 规划机制：异步 GC-Planner 决策

SIDE-CHANNEL FORK AND PLANNER TRIGGERS

围绕前缀缓存进行GC Plan

- 无侵入运行：触发 GC 条件后，主对话继续运行，系统控制层异步复制 (Fork) 当前上下文前缀。
- 追加 Planner Prompt：在 Fork 的上下文副本末尾，追加一段专门的规划 Prompt
- 输出 JSON 策略：Planner 输出形如 [FOLD/MASK/PRUNE] 的结构化指令集，并延迟安全提交。

1. 触发判定与 Fork 副本

复制主对话前缀，主流程无感

2. 追加 Planner Prompt

末尾注入特制 GC 判定指令集

3. GC-Planner 异步决策

基于元数据输出 FOLD/MASK/PRUNE

4. 延迟安全提交 (Commit)

在工具运行完/用户发新消息时更新

任务指令

请暂停其他工作，你当前正在对上述对话历史进行上下文 GC（垃圾回收）与精简规划。

必须输出且仅输出一个用 ``xml 包裹的决策块。

决策规则

1. 评估依赖：若后面的对话不再需要某段内容的原始细节，则可安全进行压缩。
2. 动作选择：
 - fold (折叠): 将大文本（如长源码、大结果）移入旁路存储，在主对话中仅留 `[Object FOLDED: ID]`
 - prune (清理): 将失败重试、冗余的工具调用完全擦除。
 - mask (遮罩): 对超长日志仅保留首尾 10%，隐藏中间。

``xml

```
<above_conversation_summary>
  [一句话纯文本总结: 对话类型与保留倾向]
</above_conversation_summary>
<gc_plan>
  <fold kind="conversation" reason="resolved_turn">
    user:5
  </fold>
  <prune kind="function" reason="obsolete_search">
    grep:3
  </prune>
</gc_plan>
```

...

裁剪动作的意图区分及尽力可恢复

CACHE-AWARE COMMIT AND LIFECYCLE ACTIONS

$$\text{Commit Value} = \text{Future Input Saving} - \text{Cache Disruption Cost}$$

Fold / Mask / Prune 区分不同价值的证据

动作	Active View 表现	Sidecar 存储	恢复保障	适用对象
Fold	[object-ref:id] 指针	保存完整原始 Byte	100% 可回填	业务报告、PDF、核心证据
Mask	[id-start] ... [id-end]	保存两端片段	部分结构恢复	DOM 快照、长文件读取结果
Prune	彻底缺失 Absent	不保存	无法恢复	失败重试日志、已 resolved 尝试

- 改动 Active View 可能破坏 suffix cache，让后续调用重新计费。
- 只有未来输入节省明确大于缓存破坏成本时才马上 Commit；否则先 Pending，等待更合适的安全轮次边界。

提交策略：延迟提交 + 增量 GC

DELAYED COMMIT + INCREMENTAL GC

两层策略，分别保护语义和缓存

1. 延迟提交：保留最后轮

Plan 先进入 pending，不在工具执行中或最新用户轮上立刻改写；等 safe turn boundary 再落地。

2. 增量 GC：稳定前缀不动

已提交、已缓存的早期前缀尽量不再改；新的 GC 主要作用在较新的对象区间。

3. 缓存破坏：只破坏尾端

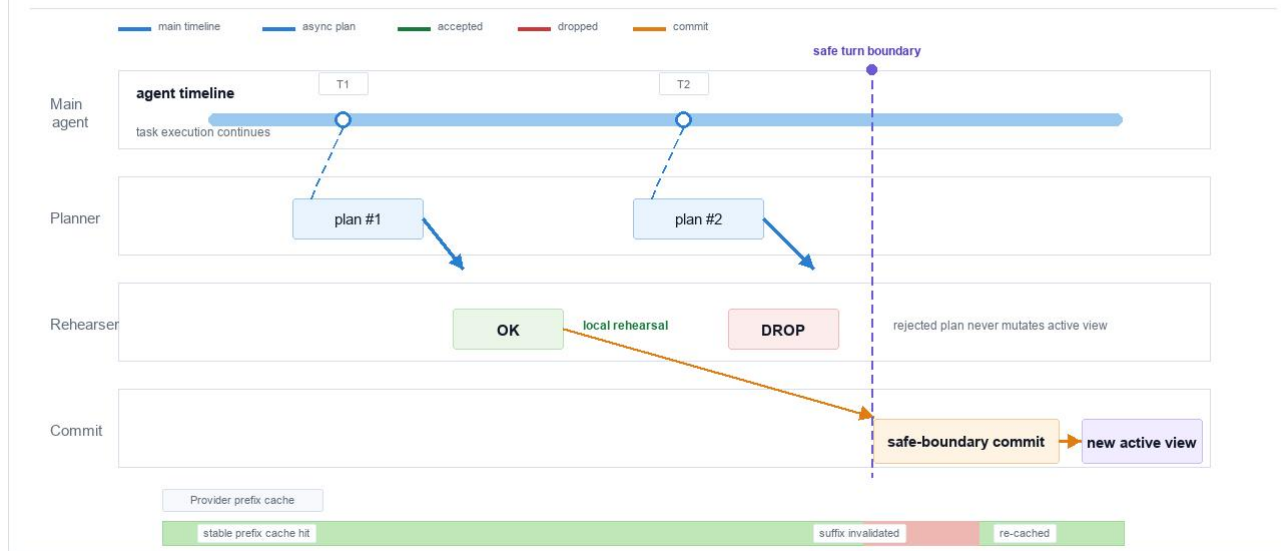
首次分叉点之前仍是 cache hit；需要重建的是被提交改写后的 suffix，而不是全量上下文。

提交条件：未来输入节省 > cache break + GC 开销
线上回归按照 30% 裁剪率作为提交阈值

一句话：延迟提交保护最后轮，增量 GC 保护 prefix cache，二者一起让上下文编辑可持续运行。

Plan-Rehearse-Commit Sequence

Side-channel plans are validated off the main loop and committed only at safe turn boundaries.



缓存影响

stable prefix cache hit

tail break

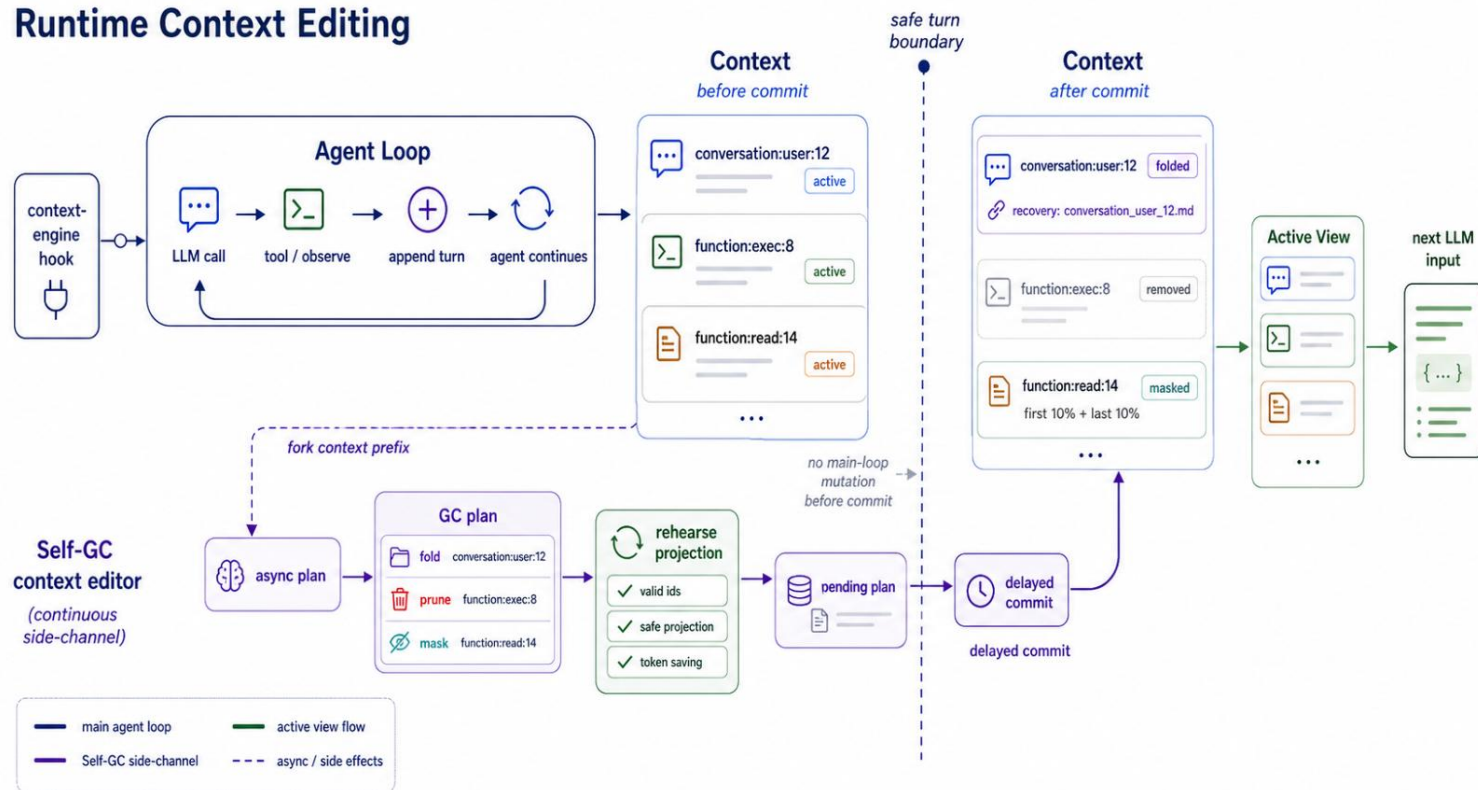
re-cached

增量 GC 的关键：分叉点尽量靠后，缓存破坏局限在尾端。

Self-GC 系统级运行架构

SIDE-CHANNEL PLAN / COMMIT

Runtime Context Editing



三阶段提交机制

- Async Plan: 触发 GC 周期后, 旁路 planner 异步 fork 当前对话前缀, 规划 Fold / Mask / Prune。
- Rehearsal: 系统控制层先本地预演, 检查 ID 幻觉、误删风险, 并计算 token saving。
- Delayed Commit: plan 先进入 pending, 等工具执行完或等待用户输入等安全边界再提交。

Planner 可以异步思考, 但上下文编辑必须由系统控制层安全提交。

Context Injection 与数据 飞轮

删除对象与注入对象是一对运行时能力



Context Injection: 旁路 Copilot 协同机制

LOW-COST SIDE-CHANNEL AGENT COLLABORATION



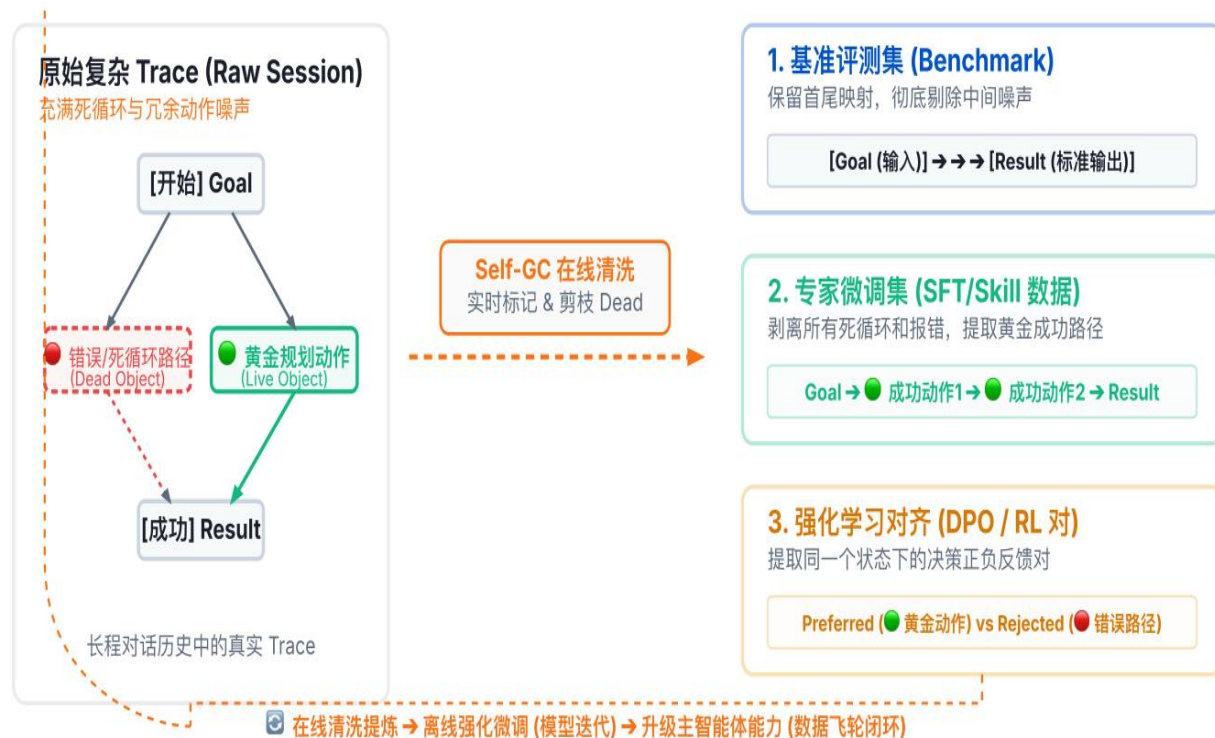
协同角色

- Master Agent: 只专注核心任务流, 负责高风险决策, 可路由到旗舰顶级模型。
- Side-channel Copilot: 独立于主执行流, 用低成本长上下文模型整理 Jira、会议、邮件、IM、Git、网页历史等背景资料。
- Context Injection: 把整理后的高浓度 ContextObject 注入 Active View, 而不是把原始材料全部塞给主智能体。
- Self-GC: 如果注入内容后续证明无用, 下一轮会把它判为 Dead Object 并回收。

注入和回收配在一起, 才能用低 Token 成本做出类似 Copilot 的协同效果。

Self-GC 作为在线 Session 数据清洗机制

SESSION CLEANING AND FEEDBACK FLYWHEEL



三类下游数据资产

- Benchmark: 保留 User Goal / Intent 与最终 Result, 去掉中间冗余探索, 形成端到端评测样本。
[Goal (输入)] → → → [Result (标准输出)]
- SFT / Skill 数据: 剥离死循环、报错和重试路径, 只留下真正走通的 Golden Trace。
Goal → 成功动作1 → 成功动作2 → Result
- DPO / RL 偏好对: 同一执行状态下, 哪些动作走向失败、哪些动作推动成功, 天然形成正负样本。
Preferred (黄金动作) vs Rejected (错误路径)
- Agent 跑得越多, Self-GC 清洗出的高质量数据越多; 数据越多, 离线微调与偏好学习越稳。

运行时垃圾回收不只是省钱工具, 也可以变成持续反哺模型的数据入口。

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



评测结果与 Workload 洞察

从 Hard Set、线上 A/B 到场景落地边界



评测方案：将裁剪变成 DIFF PATCH 评估

PRUNING RATE VS NO-IMPACT RATE

评测流程

- **真实海量线上语料**：拉取线上 70k+ 长程多轮会话（包含 BI、搜索问答、编码）为评测数据集。
- **Ground Truth 保留**：切分对话时，强制保留最后 1-2 轮真实 Turn 作为回答表现验证集。
- **裁剪动作转为Diff Patch 评测**：将 FOLD/MASK/PRUNE 的 GC 决策转化为差分补丁（Diff Patch），在裁剪后的 Active View 下重新激发模型预测最后一轮，验证模型输出一致性与任务成功率。



Hard Set: 不要只看压缩率，还要看无影响率

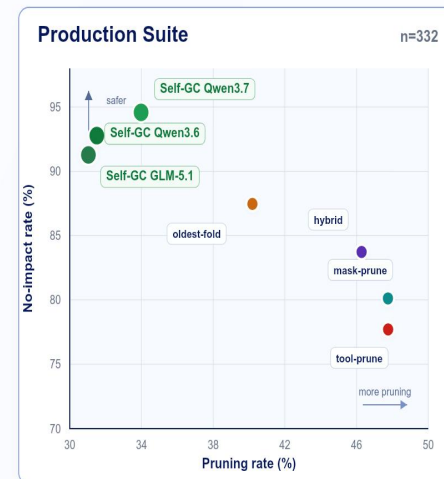
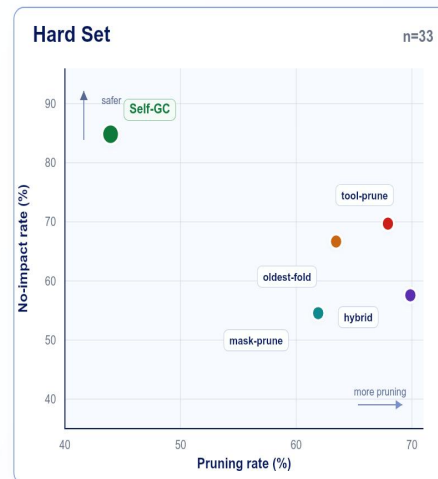
PRUNING RATE VS NO-IMPACT RATE

评测设计

- 上下文治理不能只看“压了多少”。单纯追求 Pruning Rate 很容易把关键证据也剪掉。
- 更关注双指标：Pruning Rate 与 No-Impact Rate，即剪完以后任务是否受影响。
- Hard Set 排除了 Coding 任务，主要包含长程浏览器爬取、多数据源比对、知识调研 Trace。
- 这些任务的关键证据非常稀疏，URL、页面状态或数据行被误杀时任务可能直接失败。

Compression-Preservation Trade-off

Each point is the measured operating point of one policy or planner; up is safer, right is more pruning.



Self-GC planners heuristic baselines Self-GC sits up-left: less aggressive pruning, stronger future preservation.

结果解读

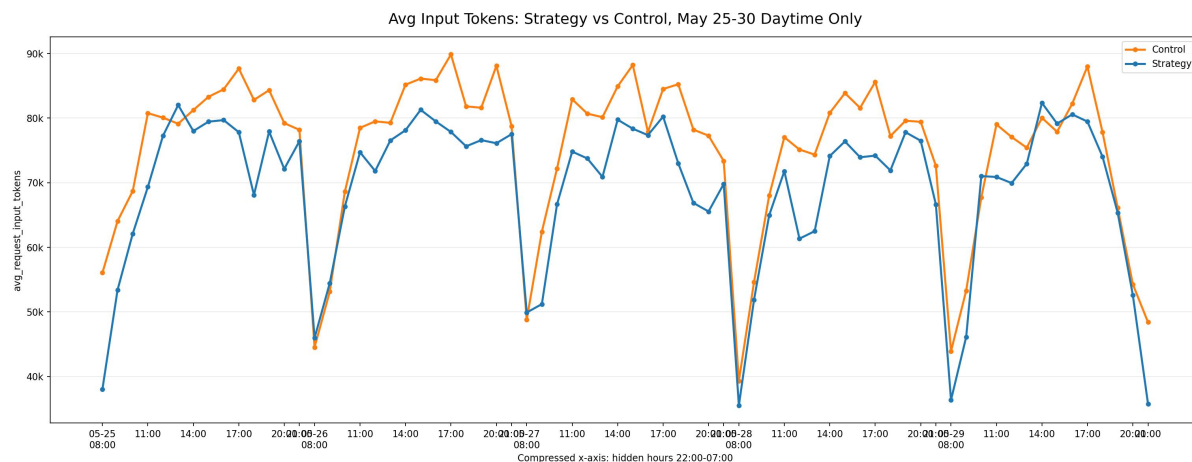
- 启发式 baseline 可把 Pruning Rate 刷到 50% 以上，但 No-Impact Rate 明显下降。
- Self-GC 在 43.95% 剪枝率下仍保持 84.85% 无影响率。
- 在 332-session 生产聚合数据上，无影响率保持在 91% 到 94%。

线上 A/B 与两类 Workload 落地洞察

ONLINE A/B AND DEPLOYMENT OBSERVATIONS

线上结果

- 小红书生产环境混合流量中，部署 Self-GC 的实验组白天平均输入 Token 下降 10% 到 15%。
- 高峰期最大降幅接近 20%，这部分下降会直接体现在云端模型费用上。



洞察一：Git 是 Coding Agent 的外部记忆基础设施，白领办公极度缺少物理兜底。

洞察二：GC Planner 不一定需要旗舰模型，中档模型往往能取得更好的成本 / 效果平衡。

Self-GC 的线上收益来自持续降低 Active View，而不是一次性压缩。

总结与展望

CONCLUSION AND FUTURE WORK

三点结论

- 窗口大不等于不用管：输入曲线下面的面积最终都会变成账单。
- 上下文不能再只是文本拼接：它需要变成可索引、可恢复、可管理生命周期的对象。
- GC 治理必须是模型判断和系统约束的闭环：模型负责判断未来价值，系统负责校验、预演、算缓存经济账，并在安全边界上提交。

当前边界与后续方向

- 当前离线评测仍依赖 LLM Judge 估计无影响率，后续需要推进基于 Replay Success 的线上端到端评测。
- 生产 trace 包含企业敏感信息，数据集需要脱敏和 fixtures sanitization。
- 当评测、数据安全和工程实现更稳定后，希望开放数据集和 Self-GC 核心引擎，让长程 Agent 上下文治理被更多团队复用。

Self-GC 的核心不是压缩文本，而是治理 Agent 运行时上下文生命周期。

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

InfoQ 极客邦科技

